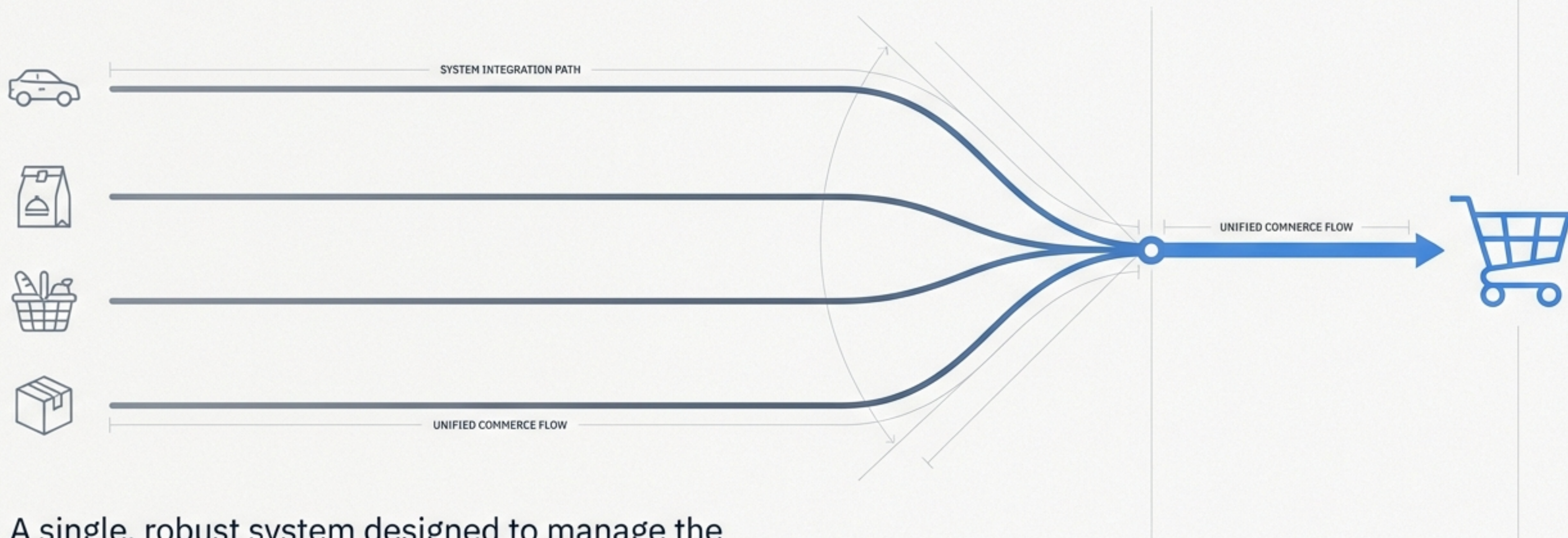


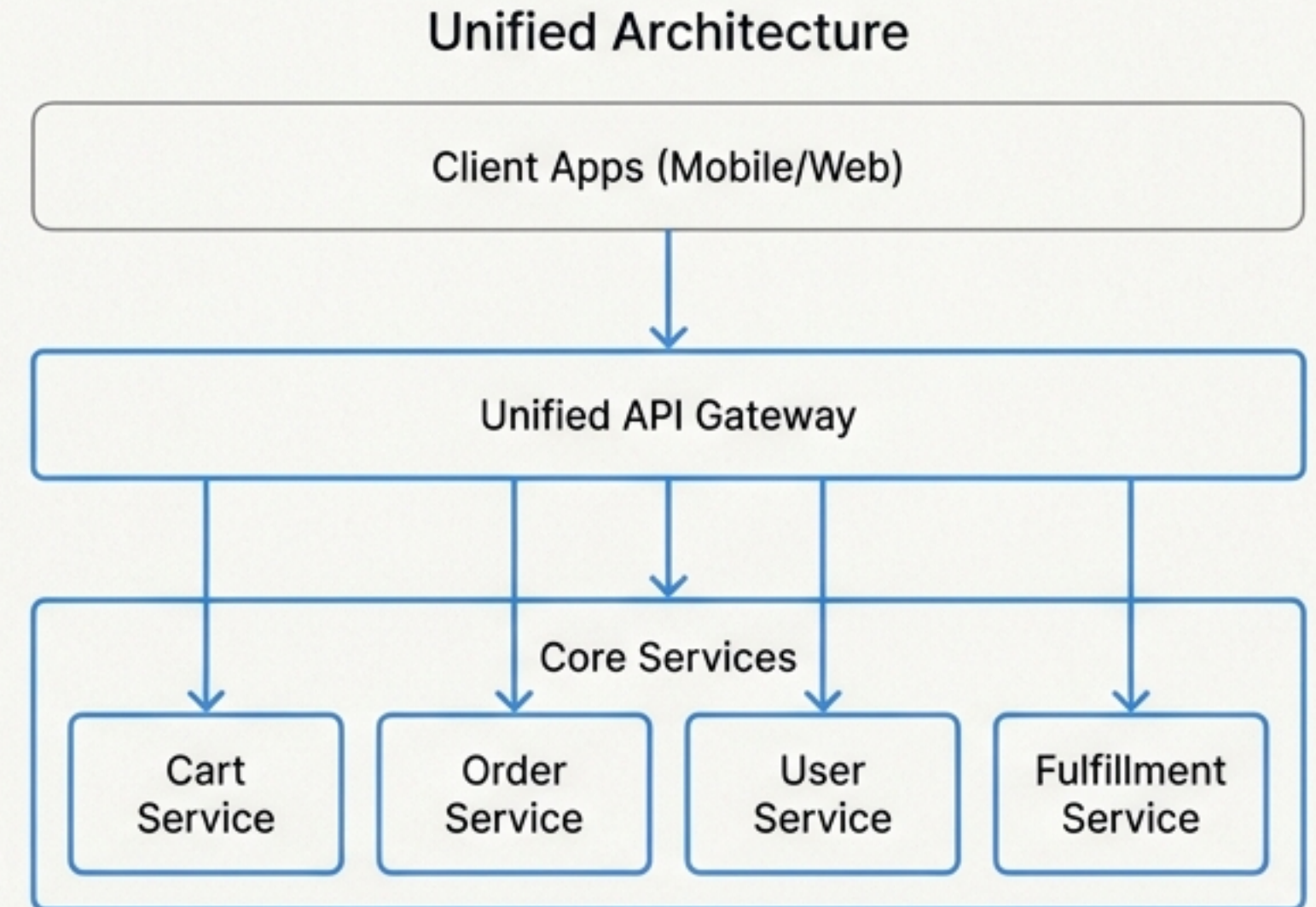
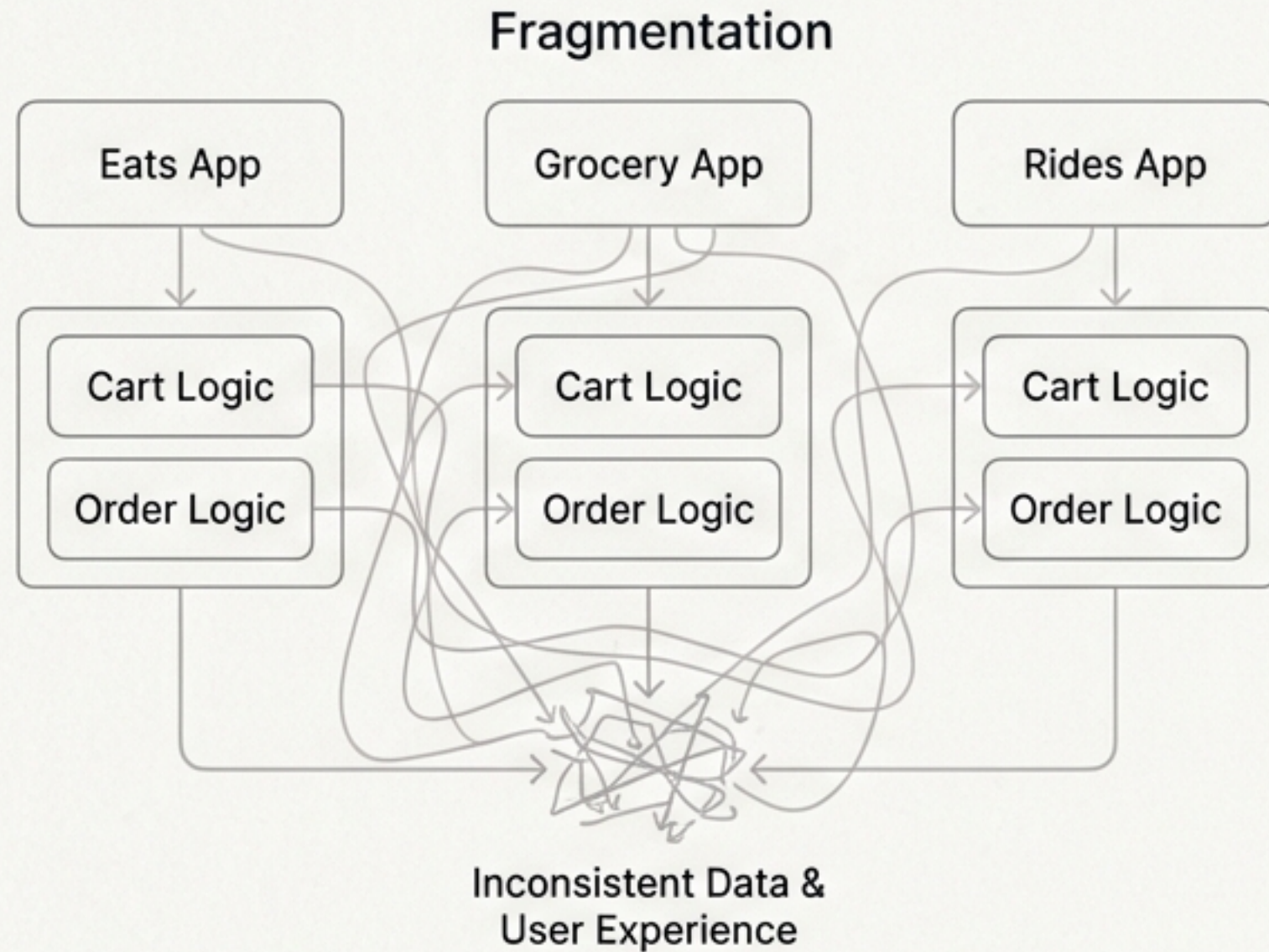
The Blueprint for Uber's Unified Commerce Experience

An Architectural Overview of the Uber Cart & Order Management System

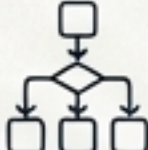


A single, robust system designed to manage the end-to-end commerce lifecycle for every Uber vertical, supporting multi-merchant carts, complex fulfillment, and a global user base.

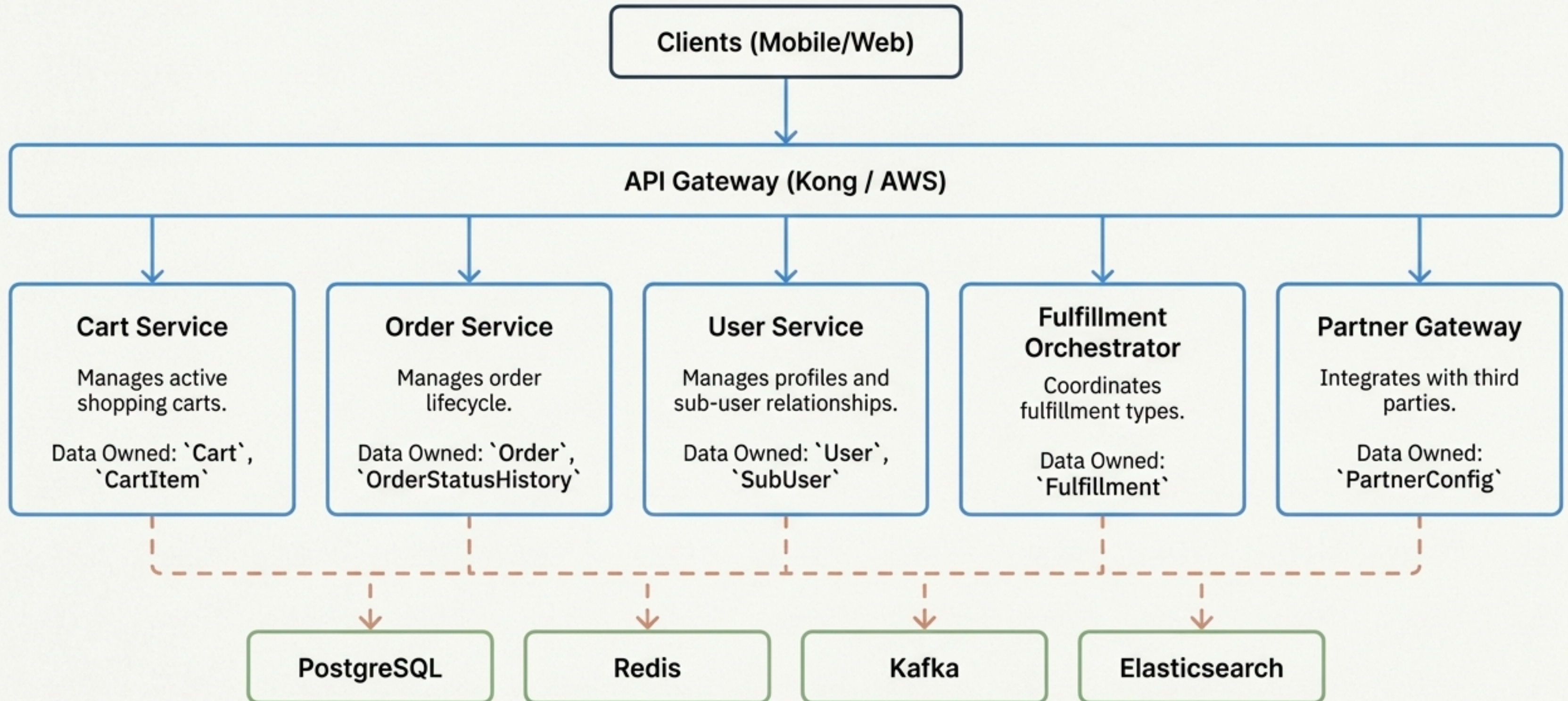
From Fragmentation to a Unified, Domain-Driven Architecture



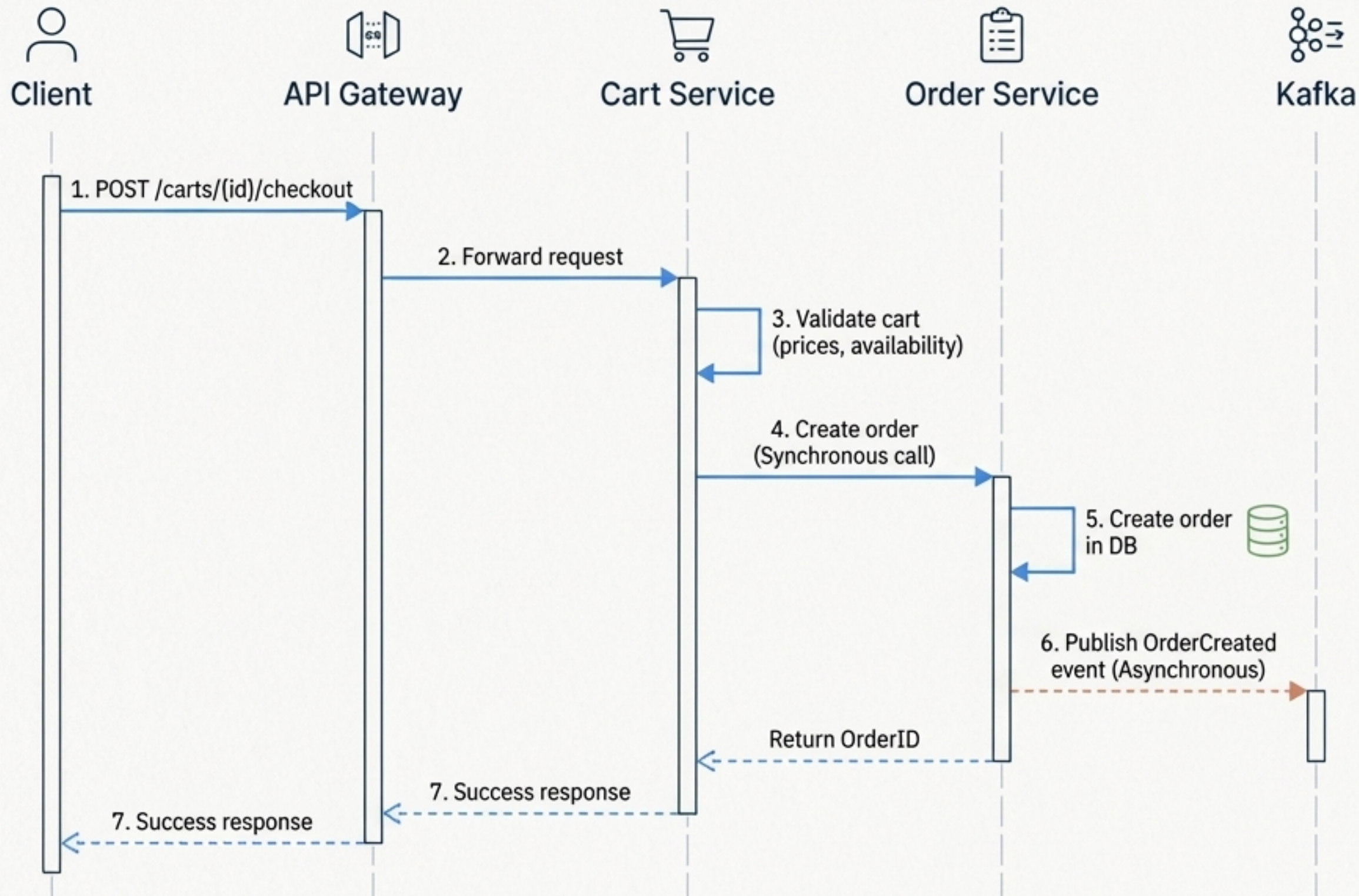
Core Architectural Principles

-  **Domain-Driven Design**
Clear bounded contexts for Cart, Order, User, and Fulfillment.
-  **Microservices Architecture**
Independently deployable services with dedicated databases.
-  **Event-Driven Communication**
Asynchronous processing for resilience and loose coupling.
-  **Offline-First Design**
A seamless user experience, regardless of connectivity.

The System Blueprint: Core Services and Their Boundaries



The Critical Path: Deconstructing the Cart-to-Order Flow



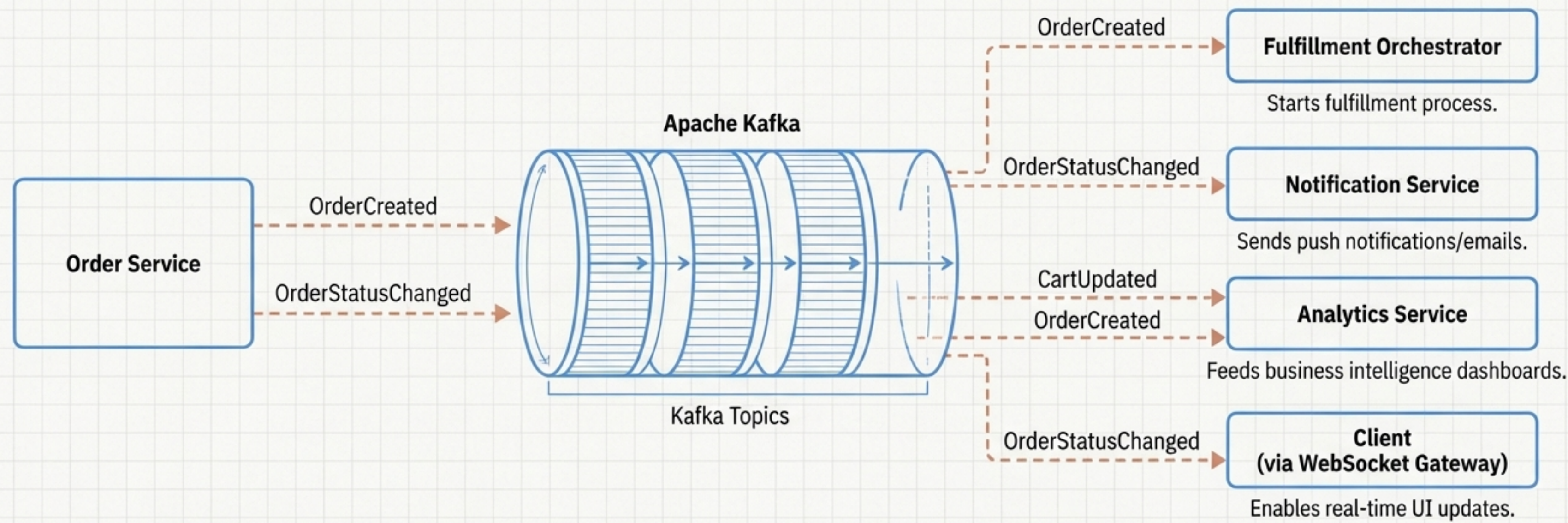
Synchronous Path (REST/gRPC)

Used for user-facing operations requiring immediate confirmation (e.g., adding to cart, checkout initiation).

Asynchronous Path (Kafka Events)

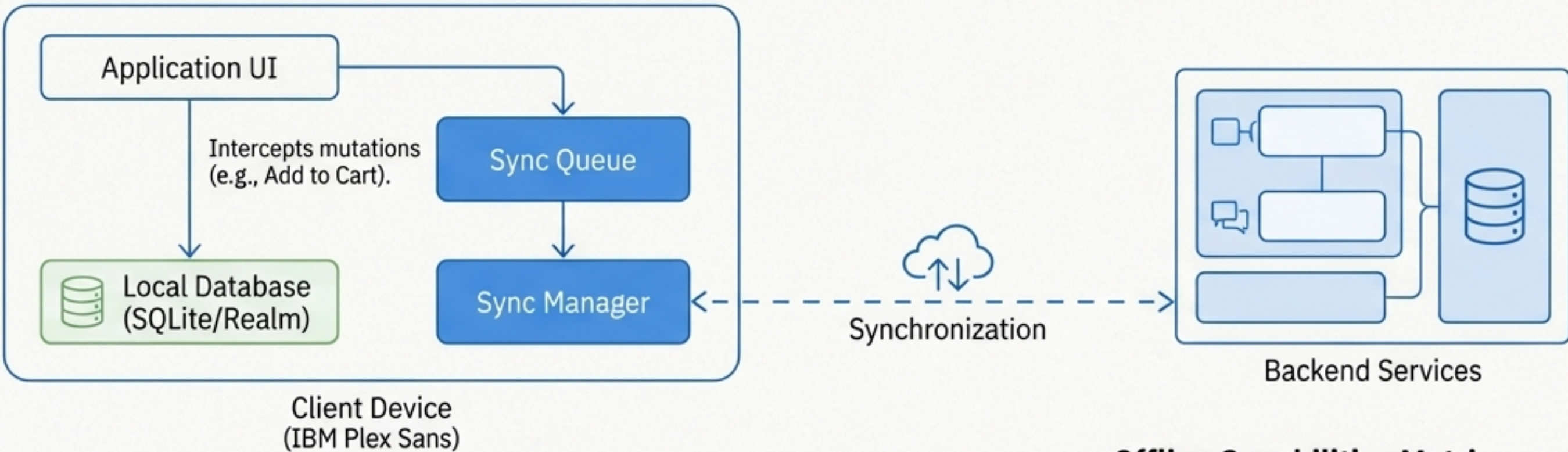
Used for post-confirmation processing like initiating fulfillment, sending notifications, and updating analytics. Ensures resilience and responsiveness.

The Event-Driven Backbone: Decoupling Services with Kafka



Event	Publisher	Key Subscribers	Purpose
OrderCreated	Order Service	Fulfillment, Notification	Initiate order processing
OrderStatusChanged	Order Service	Notification, Client	Real-time status updates
FulfillmentUpdated	Fulfillment Svc	Order Service	Update master order status
PaymentProcessed	Payment Service	Order Service	Confirm payment for order release

Pillar 1: An Offline-First Architecture for Uninterrupted Commerce



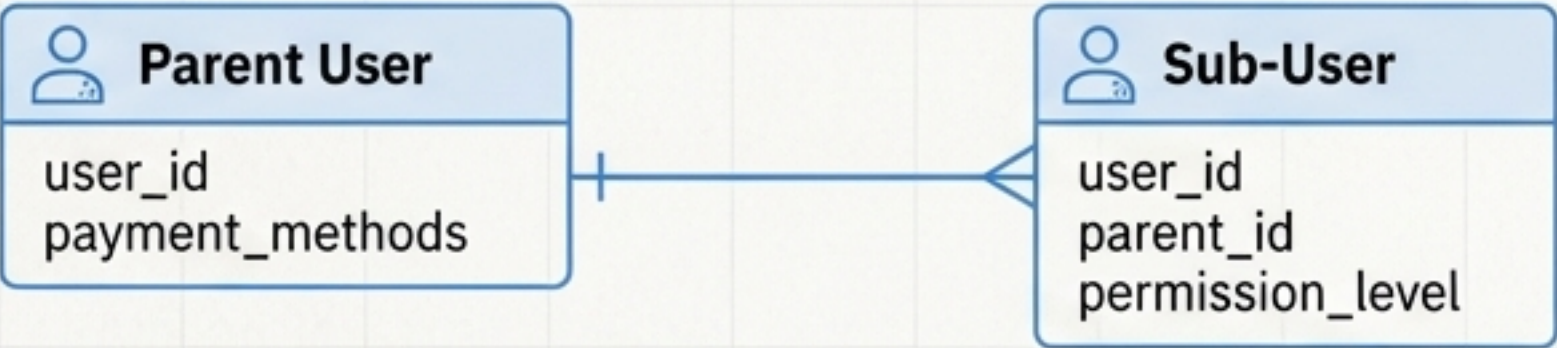
- ✓ **Local-First Operations:** All cart mutations are applied to the local database first for an **Optimistic UI**.
- ✓ **Transparent Sync:** A dedicated Sync Manager handles background data transfer and automatic conflict resolution.

Offline Capabilities Matrix

Feature	Offline Support	Notes
View/Update Cart	Full	Queued for sync
View Order History	Full	Cached locally
View Active Orders	Partial	Stale data shown with indicator
Place Order (Checkout)	None	Requires connectivity for validation
Track Real-time Order	None	Requires live data

Pillar 2: Supporting Family Accounts with Hierarchical Permissions

User Relationship Model

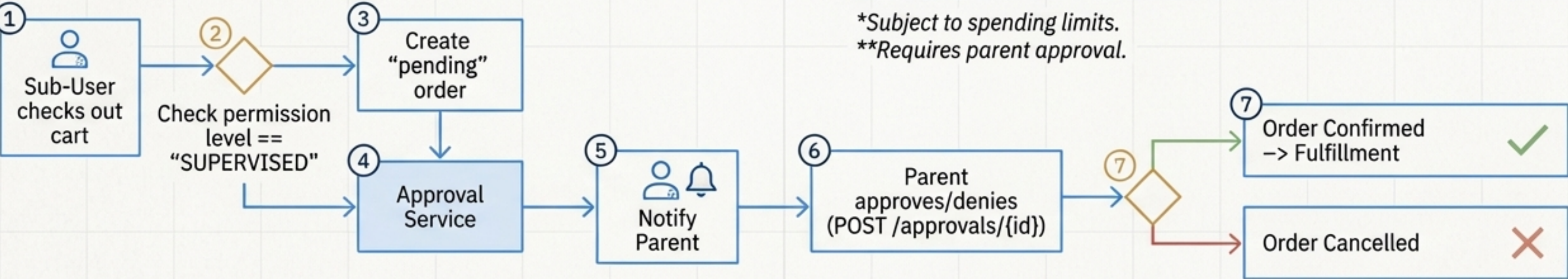


Capability Matrix

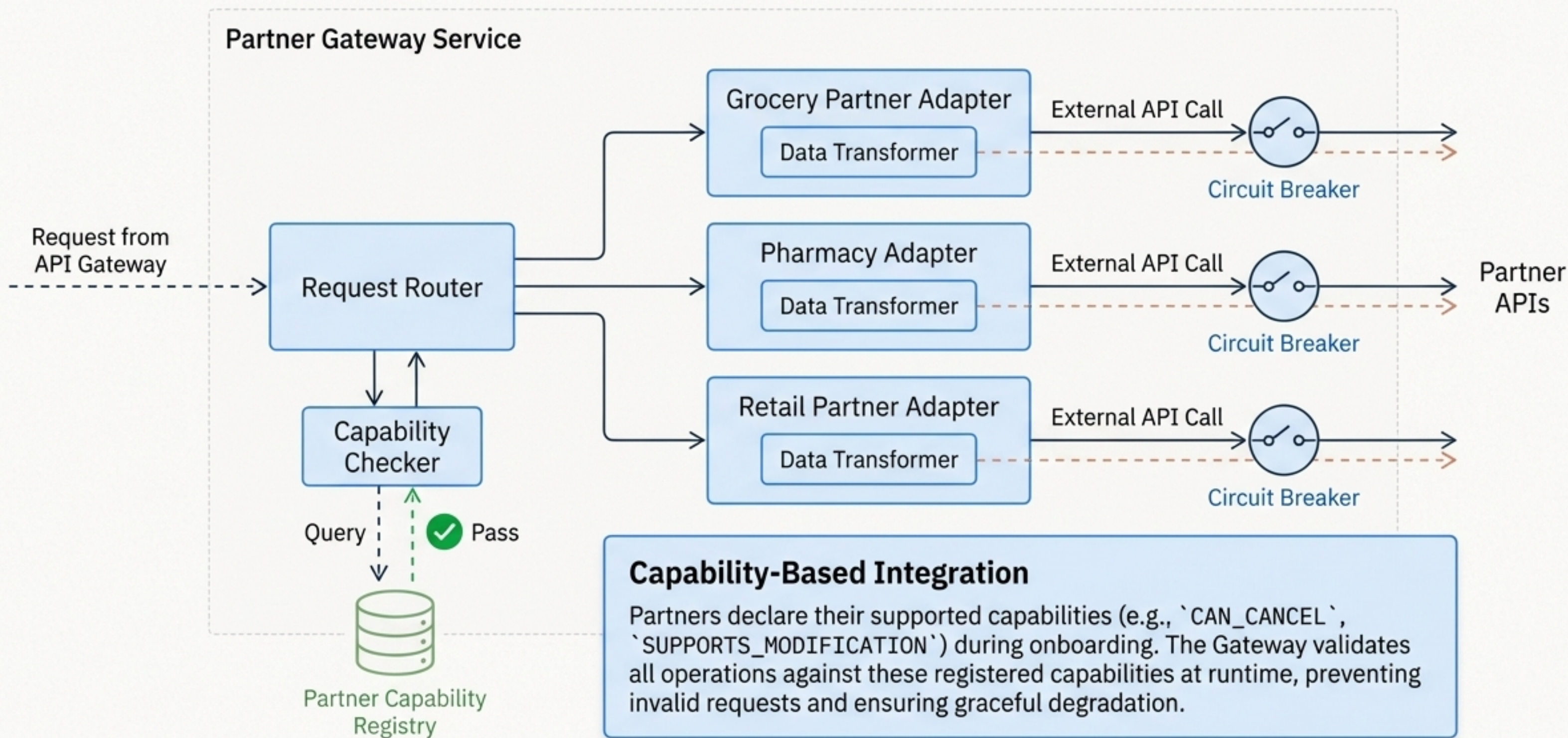
Capability	VIEW_ONLY	LIMITED	SUPERVISED	FULL
View own orders	✓	✓	✓	✓
Place orders	✗	✓*	✓**	✓
Cancel own orders	✗	✓	✓	✓
Change addresses	✗	✗	✗	✓

*Subject to spending limits.
**Requires parent approval.

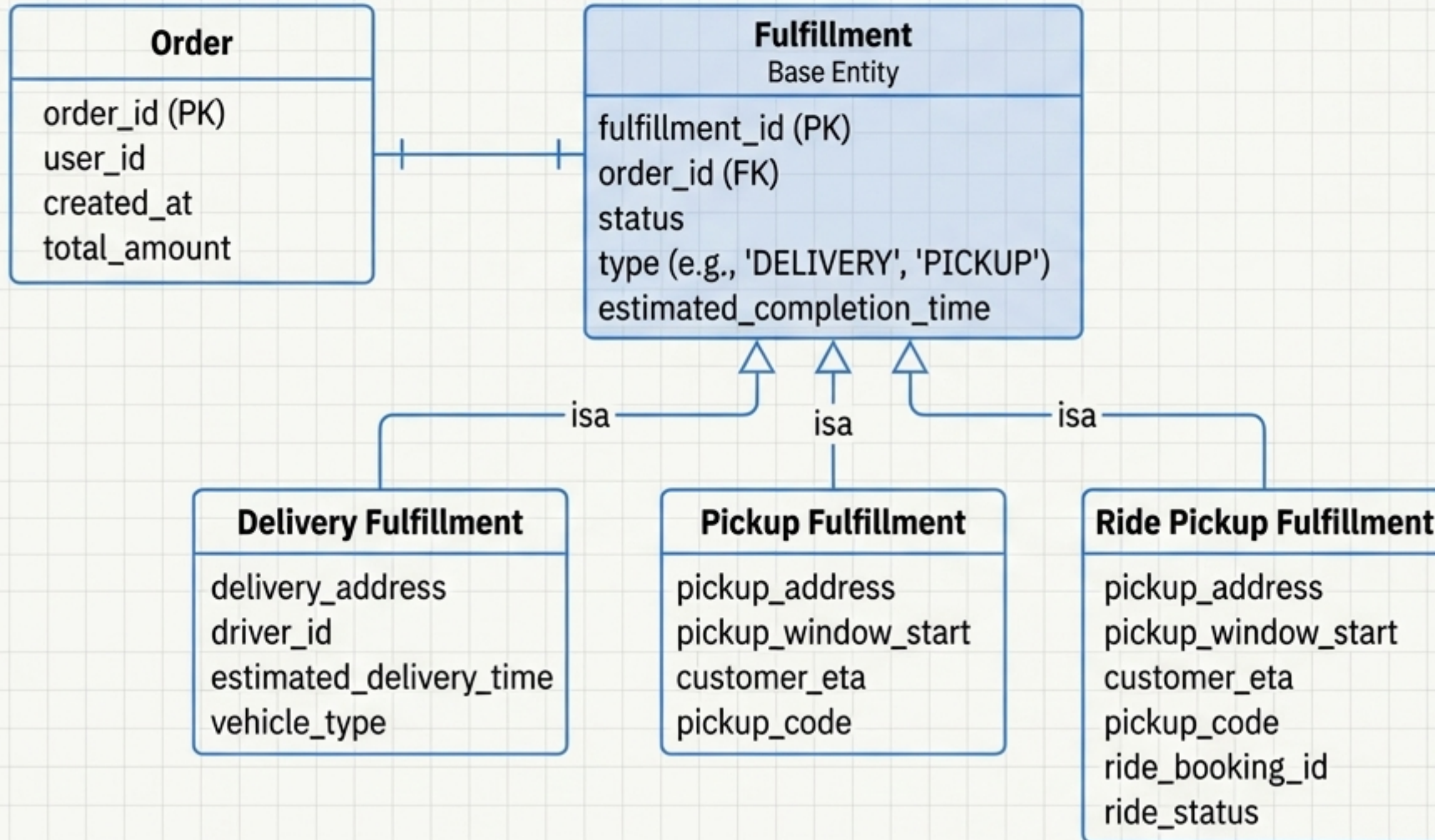
“Supervised Order” Approval Workflow



Pillar 3: A Flexible Gateway for Third-Party Partner Integration



Modeling a Polymorphic World: The Unified Fulfillment Entity



Unified Fulfillment Entity

A single, polymorphic 'Fulfillment' entity allows the Order Service to manage all fulfillment types consistently, while type-specific data is handled by specialized downstream services or stored in flexible JSONB columns.

The API Contract: A Predictable and Secure Developer Entrypoint

API Design Principles

- verb →
noun

 - RESTful Design (Nouns as resources, verbs as methods)
- URI-based Versioning (`/api/v1/`)
- Cursor-based Pagination
- Idempotency via `X-Idempotency-Key` header for mutations

Common Headers

Header	Required	Description
Authorization	Yes ✓	Bearer token (JWT)
X-Request-ID	Yes ✓	Unique request identifier for tracing
X-Idempotency-Key	On POST/PUT	Prevents duplicate mutations
X-Client-Version	Recommended	Client app version

Standard Error Response

```
{
  "request_id": "uuid-v4-goes-
  here",
  "error": {
    "code": "CART_ITEM_UNAVAILABLE",
    "message": "Item 'Avocado
    Toast' is no longer available.",
    "status": 422
  }
}
```

Key API Endpoints and Flexible Querying with GraphQL

Critical REST Endpoints

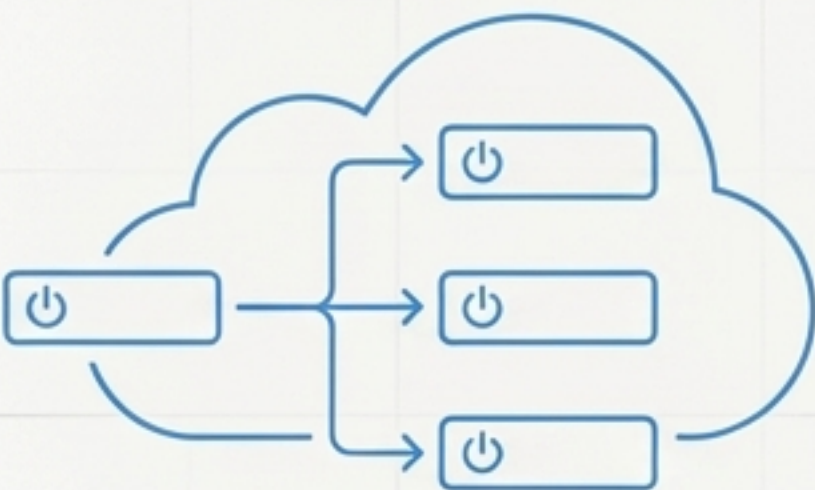
- GET** /carts/current - Get user's active cart
- POST** /carts/{id}/items - Add item to cart
- POST** /carts/{id}/checkout - Convert cart to order(s)
- GET** /orders - List user's orders
- POST** /orders/{id}/cancel - Cancel an order
- GET** /orders/family/{subUserId} - Get sub-user's orders (for parent)
- POST** /webhooks/partners/{id} - Receive partner webhooks

GraphQL for Flexible Data Fetching

```
1 query GetCartDetails {  
2   cart(id: "cart-123") {  
3     id  
4     totalPrice { amount currency }  
5     items {  
6       id  
7       name  
8       quantity  
9       merchant { name }  
10    }  
11  }  
12 }
```

A full TypeScript SDK is provided to abstract away network calls and provide type safety for client development.

Engineered for Uber Scale: Scaling, Caching, and Sharding



Horizontal Scaling

All core services are stateless and designed for horizontal scaling via Kubernetes. Service replicas are scaled independently based on real-time demand (e.g., Cart Service by active users, Order Service by order volume).



Multi-Layer Caching

Aggressive caching strategy using a Redis Cluster to reduce database load and improve latency.

Data Type	Cache TTL	Invalidation Strategy
Active Cart Data	30 min	Write-through
Menu Items	5 min	Event-based from Catalog Svc
User Preferences	1 hour	Write-through



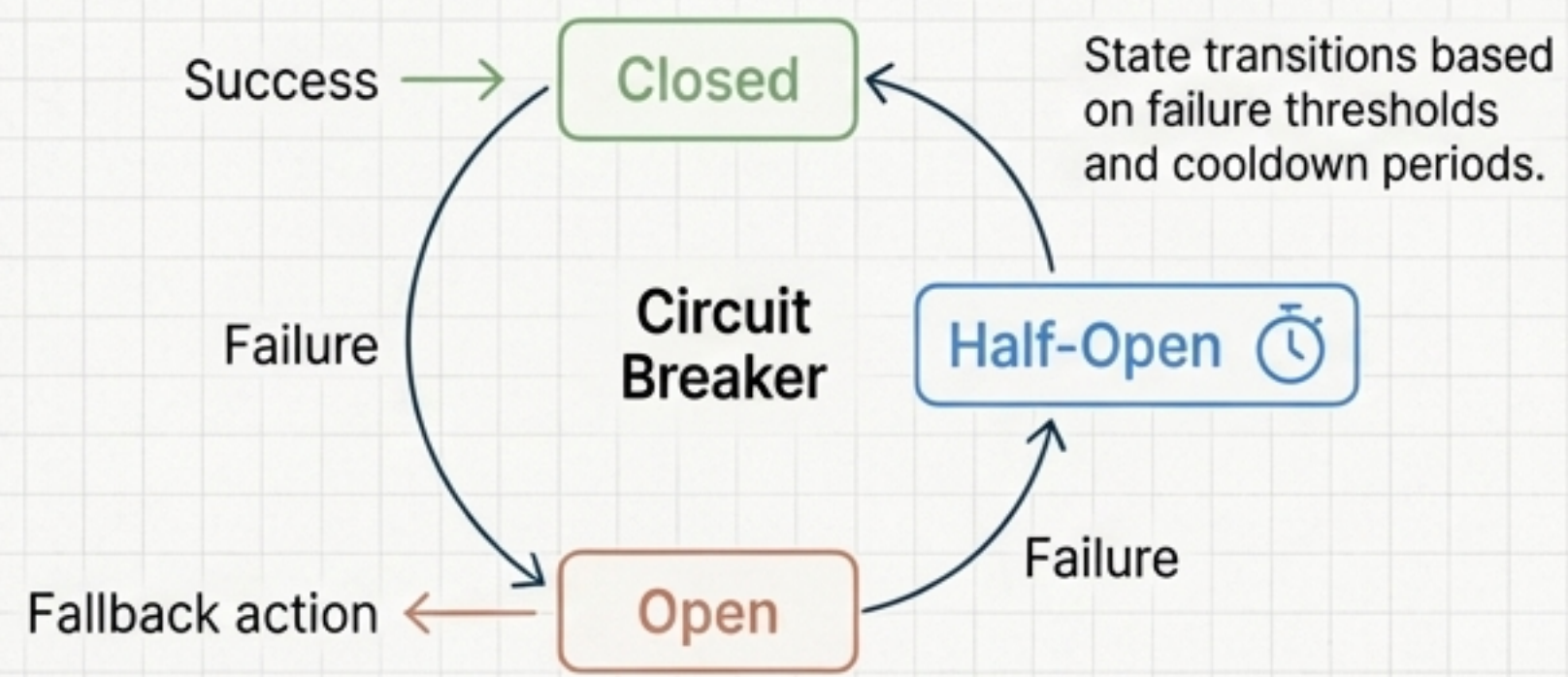
Database Sharding

Databases are sharded to distribute load and enable limitless growth.

- **Cart DB:** Sharded by ``user_id``.
- **Order DB:** Sharded by ``region`` + time-based partitioning.
- **User DB:** Sharded by ``user_id``.

Resilience by Design: Failure Handling and Fallback Strategies

Retry & Circuit Breaker Strategy



Retry Logic & Circuit Breaker Configuration

Operation	Max Retries	Backoff	Circuit Breaker
Cart Read	3	Exponential	Yes
Order Create	0 (Idempotent)	N/A	No
Fulfillment Update	5	Exponential	Yes

Key Fallback Strategies

Cart Service Down
Serve cart from client's local cache; queue write operations for later sync.

Catalog Service Down
Serve stale menu data with a UI warning to the user.

Payment Service Down
Hold the order in a 'pending payment' state; retry with exponential backoff.

The Complete Technology Stack

Layer	Technology		
Mobile/Web Client	 React Native	  Swift/Kotlin	  React + Redux Toolkit
Local Storage	 Realm	 SQLite	 IndexedDB
API Gateway	 Kong /  AWS API Gateway		
Backend Services	 Go	  Java (Spring Boot)	
Service Communication	 gRPC (internal)	REST (external)	
Primary Databases	 PostgreSQL (sharded)		
In-Memory Cache	 Redis Cluster		
Message Queue	 Apache Kafka		
Search & Analytics	 Elasticsearch		
Container Orchestration	 Kubernetes		
Service Mesh	 Istio /  Linkerd		
Distributed Tracing	 Jaeger /  Zipkin		